

APLIKACJE WEBOWE

HTTP, MVC i konfiguracja

Żądania i odpowiedzi · kody statusów
MVC · zmienne środowiskowe

L5



by ***Mateusz Węglewski***

Trzy grzechy z L4

`routes/uczniowie.js`

Wszystko w jednym pliku

Dane, logika i HTTP wymieszane w trasach

Dziś: MVC

`KLUCZ = 'tajne123'`

Sekret w kodzie

Każdy, kto widzi repozytorium, zna klucz

Dziś: plik .env

`app.listen(3000)`

Ustawienia na sztywno

Inny port na serwerze? Zmiana w kodzie

Dziś: konfiguracja

by ***Mateusz Węglewski***

Tak wygląda żądanie HTTP

```
POST /api/uczniowie HTTP/1.1
Host: localhost:3000
Content-Type: application/json
x-api-key: tajne123
↵ (pusta linia)
{"imie":"Ola","klasa":"4TI"}
```

Linia startowa: metoda · ścieżka · wersja

Nagłówki

Body (opcjonalne)

by ***Mateusz Węglewski***

A tak odpowiedź

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/uczniowie/3
Content-Length: 35
↵ (pusta linia)
{"id":3,"imie":"Ola","klasa":"4TI"}
```

Linia statusu: wersja · kod · opis **Nagłówki** **Body**

by ***Mateusz Węglewski***

LIVE DEMO 1

Podsłuchujemy HTTP

```
$ npm run dev
$ curl -v localhost:3000/api/uczniowie/1
```

› linie wysłane przez klienta
‹ linie odebrane z serwera

To samo w przeglądarce:
F12 → Network → kliknij żądanie

by ***Mateusz Węglewski***

Nagłówki, które warto znać

Nagłówek	Kierunek	Po co	W Express
Content-Type	oba	Format body	res.json() ustawia sam
Accept	żądanie	Jaki format chce klient	req.accepts('json')
x-api-key	żądanie	Kto pyta	req.get('x-api-key')
Location	odpowiedź	Gdzie jest nowy zasób	res.location(url)
Cache-Control	odpowiedź	Jak długo trzymać w cache	res.set(nazwa, wartość)
Set-Cookie	odpowiedź	Zapisz ciasteczko	res.cookie(nazwa, wartość)

by ***Mateusz Węglewski***

req i res w Express

req — co przyszło

```
req.method // 'POST'
req.params.id // z /:id
req.query.klasa // z ?klasa=
req.body // JSON
req.get('x-api-key')
req.ip
```

res — co odsyłamy

```
res.status(201) // tylko ustawia
res.json(obiekt)
res.send('tekst')
res.redirect(301, url)
res.render('widok', dane)
res.sendStatus(204)
```

Jedno żądanie = **dokładnie jedna** odpowiedź. Każda metoda poza status() i set() ją wysyła.

by ***Mateusz Węglewski***

Kody statusów: pięć rodzin

1xx	2xx	3xx	4xx	5xx
Informacja	**Sukces**	**Przekierowanie**	**Błąd klienta**	**Błąd serwera**
Rzadko spotykane	Wszystko się udało	Szukaj gdzie indziej	„Ty coś zepsułeś”	„My coś zepsuliśmy”

Pierwsza cyfra mówi, **czyja to wina**. Reszta doprecyzowuje.

by ***Mateusz Węglewski***

Kody, których użyjecie

Kod	Kiedy	W naszym API
200 OK	Sukces, zwracam dane	GET lista, GET :id
201 Created	Utworzono nowy zasób	POST
204 No Content	Sukces, nie ma czego zwrócić	DELETE
301 / 302	Przeniesiono: na stałe / chwilowo	/lista → /uczniowie
400 Bad Request	Złe lub brakujące dane	POST bez pola klasa
401 Unauthorized	Nie wiem, kim jesteś	Brak x-api-key
403 Forbidden	Wiem, kim jesteś, ale nie wolno	Uczeń usuwa oceny
404 Not Found	Nie ma takiego zasobu	/api/uczniowie/99
409 Conflict	Kolizja z tym, co już jest	Ten sam uczeń drugi raz
500 Server Error	Wyjątek po naszej stronie	Temat L6

by ***Mateusz Węglewski***

Który kod?

1 · ___

Formularz wysłany bez imienia

2 · ___

GET ucznia, którego wczoraj usunięto

3 · ___

Nowy uczeń zapisany poprawnie

4 · ___

Serwer bazy danych nie odpowiada

5 · ___

Zalogowany uczeń usuwa cudzą ocenę

6 · ___

Strona /lista na zawsze pod /uczniowie

by ***Mateusz Węglewski***

LIVE DEMO 2

Osiem kodów w dwie minuty

```
GET /api/uczniowie → 200
GET /api/uczniowie/99 → 404
POST bez klucza → 401
POST bez pola klasa → 400
POST + klucz + dane → 201
ten sam POST drugi raz → 409
DELETE /api/uczniowie/3 → 204
GET /lista → 301
```

Thunder Client w VS Code · obserwuj kolory w logu terminala

by ***Mateusz Węglewski***

Dwie klasyczne pułapki

Błąd udający sukces

```
res.json({ blad: 'Brak pól' });
// status 200 — fetch: "OK!"
res.status(400).json({ blad: … });
```

Frontend sprawdza **response.ok**, nie treść komunikatu.

Brak return

```
if (!uczen)
  res.status(404).json(…); // leci dalej!
res.json(uczen);
```

Cannot set headers after they are sent — dwie odpowiedzi na jedno żądanie.

by ***Mateusz Węglewski***

Trasa z L4 robi wszystko naraz

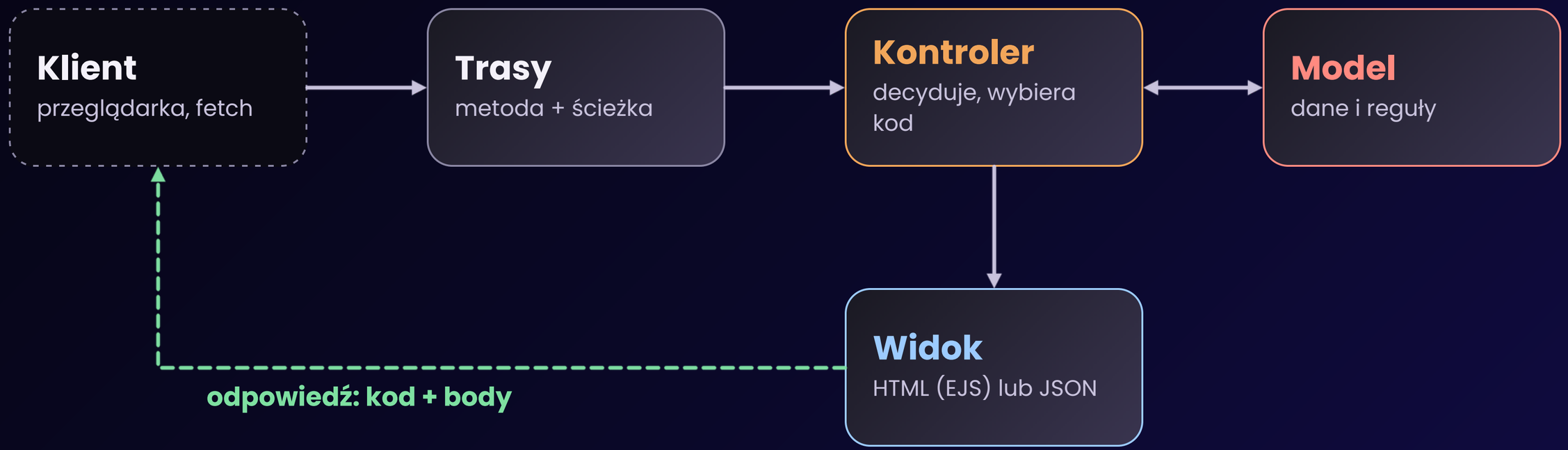
```
router.post('/', wymagajKlucza, (req, res) => {
  const { imie, klasa } = req.body;  // HTTP
  if (!imie || !klasa) return res.status(400)…  // reguła
  const nowy = { id: nastepneId++, imie, klasa };  // dane
  uczniowie.push(nowy);  // dane
  res.status(201).json(nowy);  // HTTP
});
```

Dane przenosimy do MySQL?
Przepisujemy każdą trasę.

Chcemy też stronę HTML z listą?
Kopiujemy tę samą logikę.

by ***Mateusz Węglewski***

MVC: droga żądania



by ***Mateusz Węglewski***

Kto co robi, a czego nie

Warstwa	Odpowiada za	Nigdy nie	Plik
Model	Dane i reguły dostępu	dotyka req i res	models/uczen.model.js
Widok	Wygląd strony	pobiera danych sam	views/uczniowie.ejs
Kontroler	HTTP ↔ model, kod statusu	przechowuje danych	controllers/...controller.js
Trasy	Metoda + ścieżka → funkcja	zawiera logiki	routes/...routes.js

Test: czy plik modelu da się użyć **bez Expressa**? Jeśli tak — podział jest dobry.

by ***Mateusz Węglewski***

Ten sam POST po podziale

routes/…routes.js

```
router.post('/',
  wymagajKlucza,
  ctrl.dodaj);
```

controllers/…controller.js

```
exports.dodaj = (req, res) => {
  // walidacja → 400, 409
  const nowy =
    Uczen.dodaj(req.body);
  res.status(201).json(nowy);
};
```

models/uczen.model.js

```
function dodaj(dane) {
  const nowy =
    { id: nextId++, ...dane };
  uczniowie.push(nowy);
  return nowy;
}
```

Tablica → MySQL? Zmieniamy **tylko model**. Trasy i kontroler zostają.

by ***Mateusz Węglewski***

LIVE DEMO 3

Jeden model, dwie odpowiedzi

kontroler

```
// /api/uczniowie → JSON
res.json(Uczen.wszyscy());

// /uczniowie → HTML
res.render('uczniowie',
  { uczniowie: Uczen.wszyscy() });
```

views/uczniowie.ejs

```
<ul>
<% uczniowie.forEach(u => { %>
  <li><%= u.imie %></li>
<% }) %>
</ul>
```

<% %> wykonaj kod · **<%= %>** wstaw wartość (bezpiecznie, z escapowaniem)

by ***Mateusz Węglewski***

Konfiguracja aplikacji

app.js — ustawienia

```
app.set('view engine', 'ejs');
app.set('views', './views');
app.disable('x-powered-by');
app.locals.nazwaSzkoly = config.nazwaSzkoly;

if (config.env === 'development')
  app.use(logger);
```

server.js — tylko start

```
const app =
  require('./app');
app.listen(config.port);
```

Testy importują **app** bez uruchamiania serwera.

by ***Mateusz Węglewski***

Ten sam kod, różne środowiska

Twój laptop

```
PORT=3000
NODE_ENV=development
API_KEY=tajne123
```

Serwer szkolny

```
PORT=8080
NODE_ENV=production
API_KEY=k7Qp…x92
```

Kod aplikacji

```
process.env.PORT
process.env.NODE_ENV
process.env.API_KEY
```

Zasada: **konfiguracja żyje w środowisku, nie w kodzie**.

by ***Mateusz Węglewski***

Plik .env i jego bliźniak

.env — NIE do Gita

```
PORT=3000
NODE_ENV=development
API_KEY=tajne123
```

.env.example — DO Gita

```
PORT=3000
NODE_ENV=development
API_KEY=wpisz-swoj-klucz
```

```
# .gitignore
.env
# package.json — wbudowane w Node 20.6+, bez paczki dotenv
"dev": "node --env-file=.env --watch server.js"
```




by ***Mateusz Węglewski***

config.js — jedno miejsce

```
function wymagana(nazwa) {
  const w = process.env[nazwa];
  if (!w) {
    console.error(`Brak ${nazwa}`);
    process.exit(1);
  }
  return w;
}
module.exports = {
  port: Number(process.env.PORT) || 3000,
  apiKey: wymagana('API_KEY'),
};
```

process.env czytamy tylko tutaj

Wszystko to **stringi** — stąd Number()

Opcjonalna → **wartość domyślna**

Wymagana → **fail fast** przy starcie

by ***Mateusz Węglewski***

LIVE DEMO 4

Psujemy konfigurację

#	Robimy	Widzimy
1	Zmieniamy nazwę .env i startujemy	node: .env: not found
2	Usuwamy linię API_KEY	Brak zmiennej API_KEY... i wyjście
3	W .env wpisujemy PORT=4000	Serwer na :4000, kod bez zmian
4	NODE_ENV=production npm start	Cisza w logach, /api/status
5	git status	.env nie ma na liście

by ***Mateusz Węglewski***

Sekret w repozytorium to sekret publiczny.

Wyciekł? Nie wystarczy usunąć commita.
Unieważnij klucz i wygeneruj nowy.

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

:wq

L5



by ***Mateusz Węglewski***